# Stack Implementation

Data Structures and Algorithms

Muhammad Rayhan
2086022510019
IMT
Universitas Ciputra Surabaya (Kampus Kota Makassar)
mrayhan02@student.ciputra.ac.id

ACC
Keihan Pradha Muzaki
10 Maret 2026

## I. Bracket Verification

### 1.1. Problem Summary

Soal ini berfokus pada verifikasi bracket kurung yang akan memberikan output *Boolean* berupa *true* atau *false* jika bracket kurung tersebut memiliki pasangan nya (contoh karakter '(' harus berpasangan dengan ')' agar outputnya menjadi *true*)

### 1.2. Stack Design

What gets pushed/popped and why: Hanya karakter tanda kurung pembuka (misalnya (, {, [ ) yang akan di-*push* ke dalam Stack. Ketika sistem mendeteksi karakter tanda kurung penutup (misalnya ), }, ]), elemen pada puncak Stack akan di-*pop* untuk mencocokan jenis kurungnya.

How the stack represents the current state: Stack bertindak sebagai memori historis yang merepresentasikan daftar tanda kurung pembuka yang sedang "menunggu" untuk ditutup.

### 1.3. Algorithm Explanation

Step-by-step logic: Program melakukan iterasi secara linier pada setiap karakter dalam input. Jika karakter adalah pembuka, maka akan di-*push* ke dalam Stack. Jika karakter adalah penutup, program me-*pop* elemen teratas dari Stack dan membandingkannya. Jika pasangan cocok, iterasi berlanjut. Di akhir iterasi, jika Stack dalam keadaan kosong, sistem mengembalikan true.

Handling of edge cases / invalid states: Algoritma menangani dua kondisi *invalid* utama secara langsung: (a) mendeteksi kurung penutup saat Stack sudah kosong (kasus kelebihan penutup), dan (b) Stack masih menyisakan elemen setelah seluruh *string* selesai diproses (kasus kelebihan pembuka). Keduanya secara instan akan memicu pengembalian nilai false.

### 1.4. Correctness Argument

Why the algorithm produces the required output: Sifat *Last-In, First-Out* (LIFO) pada struktur data Stack menjamin bahwa tanda kurung pembuka yang terakhir kali dimasukkan akan menjadi entitas pertama yang dicarikan pasangannya. Mekanisme ini secara matematis merepresentasikan hukum persarangan hierarki yang benar.

### 1.5. Time & Space Complexity

Time Complexity: O(N), di mana N adalah jumlah karakter dalam *string*. Setiap karakter hanya diproses maksimal satu kali dengan operasi *push* atau *pop* yang memiliki waktu konstan O(1).

Space Complexity: O(N) pada *worst-case scenario* (misalnya *string* hanya berisi tanda kurung pembuka ((((((), karena seluruh karakter tersebut akan dialokasikan ke dalam memori Stack.

### 1.6. Test Evidence

Testcase 1 {[()]}
Custom case 1 ([)]
Custom case 2 {[]}

Output (untuk ke-3 testcase):
*True*
*False*
*True*

### 1.7. Code Reference

- Source Code File

https://github.com/HannzzIDK/DSA-Stack-MuhammadRayhan-2086022510019/blob/main/src/Question1.java

## II. Reverse Polish Notation

### 2.1. Problem Summary

Soal ini berfokus pada penyelesaian permasalahan matematika menggunakan pendekatan *reverse polish notation* atau *postfix notation* yang dimana sedikit berbeda dengan metode aritmatika sederhana, notasi ini menggunakan penanda operasi di akhir (misal +,_,*,/) namun dengan hasil yang tetap sama dengan aritmatika pada umumnya.

### 2.2. Stack Design

What gets pushed/popped and why: pada kasus ini angka di depan operasi yang akan di push, jika stack sudah mendeteksi operator (misal +,_,*,/), stack akan me-*pop* dua elemen teratasnya, dan hasil operasi tersebut di *push* kembali ke dalam stack

How the stack represents the current state: Stack bertindak sebagai tempat untuk menyimpan ke dua bilangan yang akan di operasikan ke dalam stack, hal ini diperlukan dalam

XXX-X-XXXX-XXXX-X/XX/$XX.00 ©20XX IEEE

Dipindai dengan CamScanner

permasalahan seperti ini karena jika nilai tidak di simpan maka output program akan berupa error karena kesalahan logika.

2.3. Algorithm Explanation

Step-by-step logic: Algoritma akan melakukan pemindaian linier pada setiap token. Jika token merupakan angka, maka langsung dimasukkan ke Stack. Jika token adalah operator, sistem mengambil dua elemen terakhir, menghitung hasilnya, dan mengembalikan hasilnya ke puncak tumpukan. Proses berakhir ketika hanya tersisa satu elemen di Stack (berupa hasil akhir).

Handling of edge cases / invalid states: Jika operator ditemukan namun Stack kurang dari dua elemen, program akan memicu pengecekan EmptyStackException untuk mencegah *runtime error*. Selain itu, di akhir evaluasi, sistem memastikan hanya terdapat tepat satu elemen tersisa di dalam Stack, jika terdapat sisa elemen lain, maka ekspresi dinyatakan sebagai *invalid expression*.

2.4. Correctness Argument

Why the algorithm produces the required output: Algoritma menjamin ketepatan hasil karena mengikuti prinsip asosisatif notasi *Postfix*.Stack memastikan hanya angka yang paling dekat dengan operator diproses terlebih dahulu sesuai aturan presedensi manual.

2.5. Time & Space Complexity

- Time Complexity: O(N) karena setiap token hanya diproses satu kali.
- Space Complexity: O(N) untuk menyimpan tumpukan angka sebelum dihitung.

2.6. Test Evidence

Testcase 1: 21+3*
Custom case 1: 12+3*45+6/7-
Custom case 2: 12+3*45+6/7-8

Output (untuk ke-3 testcase):
*9*
*-6*
*-48*

2.7. Code Reference

- Source Code File

https://github.com/HannzzIDK/DSA-Stack-MuhammadRayhan-2086022510019/blob/main/src/Question2.java

## III. CHRONO STACK ENGINE

3.1. Problem Summary

Soal ini focus pafa sistem *Chrono Engine* yang membangun nilai target melalui instruksi 1, d, dan +. Tantangan utamanya adalah adanya efek "distorsi waktu", di mana setiap instruksi + akan mengurangi nilai elemen di bawahnya dalam tumpukan sebesar satu poin.

3.2. Stack Design

What gets pushed/popped and why: Stack digunakan untuk menyimpan urutan angka input yang diberikan pengguna. Pemrosesan dilakukan secara LIFO (dari angka terakhir kembali ke angka pertama) untuk menghitung akumulasi distorsi secara progresif.

How the stack represents the current state: Stack merepresentasikan *timeline* kronologis dari atas ke bawah, di mana elemen terbawah adalah yang paling terdampak oleh distorsi akumulatif.

3.3. Algorithm Explanation

Step-by-step logic: program memakai metode *Top-Down Distortion Compensation.* Algoritma akan menghitung rute pembentukan setiap angka secara mundur menggunakan logika ganjil/genap. Setiap kali tanda + ditemukan dalam rute elemen atas, nilai akumulasiDistorsi bertambah, yang kemudian ditambahkan ke target angka di bawahnya sebelum rute angka tersebut dicari.

Handling of edge cases / invalid states: Kasus batas ditangani dengan memvalidasi nilai target awal. Jika pengguna memasukkan angka 0, sistem secara otomatis mengabaikan perhitungan rute karena target sudah tercapai. Selain itu, algoritma dirancang untuk tetap konsisten walaupun terjadi akumulasi distorsi yang besar, di mana setiap *state* transisi selalu diperiksa agar tidak menghasilkan nilai negatif sebelum rute instruksi 1, d, atau + ditentukan.

3.4. Correctness Argument

Why the algorithm produces the required output: Metode kompensasi proaktif ini memastikan bahwa nilai akhir setiap elemen akan tepat sesuai target setelah terpotong oleh distorsi di masa depan.

3.5. Time & Space Complexity

- Time Complexity: O(N), di mana N adalah jumlah karakter dalam *string*. Setiap karakter hanya diproses maksimal satu kali dengan operasi *push* atau *pop* yang memiliki waktu konstan O(1).
- Space Complexity: O(N) pada *worst-case scenario* (misalnya *string* hanya berisi tanda kurung pembuka ((((((, karena seluruh karakter tersebut akan dialokasikan ke dalam memori Stack.

Dipindai dengan CamScanner

3.6. Test Evidence

- Testcase 1
  Input : 5 : 1 5 6 3 2
  Output
  1d+1+d+d+1d+1+d+d+1d+d+d+1+1d+d+1d+
- Custom Case 1
  Input: 4: 3 2 4 1
  Output : 1d+1+d+1+1d+d+1d+d+1
- Custom case 2
  Input : 3: 4 5 2
  Output : 1d+d+d+1d+1+d+1d+

3.7. Code Reference

- Source Code File

https://github.com/HannzzIDK/DSA-Stack-MuhammadRayhan-2086022510019/blob/main/src/Question3.java

## IV. CHEAT BUSTER SECURITY SYSTEM

4.1. Problem Summary

Soal ini berfokus pada Sistem *Cheat Buster* memerlukan mekanisme pengurutan data (*sorting*) tanpa akses indeks acak. Seluruh operasi harus dilakukan melalui manipulasi puncak tumpukan Stack untuk mencapai urutan *ascending*.

4.2. Stack Design

What gets pushed/popped and why: Elemen di-*pop* dari mainStack ke dalam *buffer* (tempArray) jika nilainya lebih besar dari elemen baru. Elemen baru kemudian di-*push*, diikuti dengan pengembalian seluruh isi *buffer* ke dalam tumpukan utama.

How the stack represents the current state: mainStack bertindak sebagai penyimpanan data yang selalu dalam kondisi sorted.

4.3. Algorithm Explanation

Algoritma ini sangat bergantung pada pemeriksaan !mainStack.isEmpty() di setiap iterasi. Hal ini mencegah terjadinya *error* saat membandingkan elemen baru dengan puncak Stack yang masih kosong (pada input pertama). Selain itu, penanganan elemen duplikat (angka yang sama) dilakukan dengan membiarkan angka tersebut di-*push* tanpa harus membongkar Stack, karena kondisi mainStack.peek() > angkaBaru tidak akan terpenuhi, sehingga efisiensi tetap terjaga.mengembalikan true.

Handling of edge cases / invalid states: Algoritma menangani dua kondisi *invalid* utama secara langsung: (a) mendeteksi kurung penutup saat Stack sudah kosong (kasus kelebihan penutup), dan (b) Stack masih menyisakan elemen setelah seluruh *string* selesai diproses (kasus kelebihan pembuka). Keduanya secara instan akan memicu pengembalian nilai false.

4.4. Correctness Argument

Why the algorithm produces the required output: Algoritma menjamin data terurut karena setiap elemen baru hanya menetap di Stack jika tidak ada elemen yang lebih besar di atasnya.

4.5. Time & Space Complexity

Time Complexity: O(N^2), sebagai kasus terburuk jika data yang ingin di urutkan terbalik.
Space Complexity: O(N) untuk menyimpan string dan buffer

4.6. Test Evidence

- Testcase 1 :
  Input 42 9 17 63 28 5 74
  Output 5 9 17 28 42 63 74
- Custom case 1:
  Input 3 5 1 2 7
  Output 1 2 3 5 7
- Custom case 2
  Input 9 34 2 5 7 43
  Output 2 5 7 9 34 43

4.7. Code Reference

- Source Code File

https://github.com/HannzzIDK/DSA-Stack-MuhammadRayhan-2086022510019/blob/main/src/Question4.java

## V. CHASTICE OPERATION

5.1. Problem Summary

Soal ini berfokus pada Simulasi *Operation Chastise* menghitung degradasi momentum bom pantul yang berkurang sebesar 50% di setiap pantulan hingga mencapai nilai minimal 1. Hasil harus ditampilkan dalam urutan keatas (dari momentum terkecil ke momentum awal).

5.2 Stack Design

What gets pushed/popped and why Nilai momentum hasil kalkulasi di-*push* ke Stack selama nilainya > 0 Karena perhitungan menghasilkan urutan menurun (50, 25, 12...), Stack digunakan untuk membalikkan urutan tersebut saat pencetakan.

How the stack represents the current state: Stack bertindak untuk membalik urutan.

5.3. Algorithm Explanation

Algoritma melakukan pembagian dua secara berulang pada input momentum menggunakan tipe data Integer. Setiap hasil pembagian dimasukkan ke Stack. Setelah momentum mencapai nol, seluruh isi Stack di-*pop* satu per satu untuk menghasilkan deret menaik

Handling of edge cases / invalid states: Sistem menangani kasus input momentum bernilai 0 atau negatif dengan menggunakan kondisi while (momentum > 0). Jika input awal tidak memenuhi syarat tersebut, Stack akan tetap kosong dan program tidak akan mencetak deret angka apa pun, yang mana merupakan respon tepat untuk objek tanpa

Dipindai dengan CamScanner

momentum. Penggunaan tipe data Integer juga secara otomatis menangani batas bawah momentum, di mana hasil pembagian 1 / 2 akan menghasilkan 0, yang secara instan menghentikan siklus pemantulan bom.

5.4. Correctness Argument

Why the algorithm produces the required output: Algoritma menjamin data terurut karena setiap elemen baru hanya menetap di Stack jika tidak ada elemen yang lebih besar di atasnya.

5.5. Time & Space Complexity

Time Complexity: O(N^2), sebagai kasus terburuk jika data yang ingin di urutkan terbalik

Space Complexity: O(N) untuk menyimpan string dan buffer

5.6. Test Evidence

- Testcase 1 :
  Input 50
  Output 1 3 6 12 25 50
- Custom case 1:
  Input 40
  Output 1 2 5 10 20 40
- Custom case 2
  Input 30
  Output 1 3 7 15 30

5.7. Code Reference

- Source Code File

https://github.com/HannzzIDK/DSA-Stack-MuhammadRayhan-2086022510019/blob/main/src/Question5.java

Dipindai dengan CamScanner